[Huyen_Nguyen] Linux OS Overview & Kernel Module/Driver Basics

1. Linux OS Overview

An OS is system software that acts as an intermediary bridge between hardware (CPU, RAM, etc.) and the user/applications.

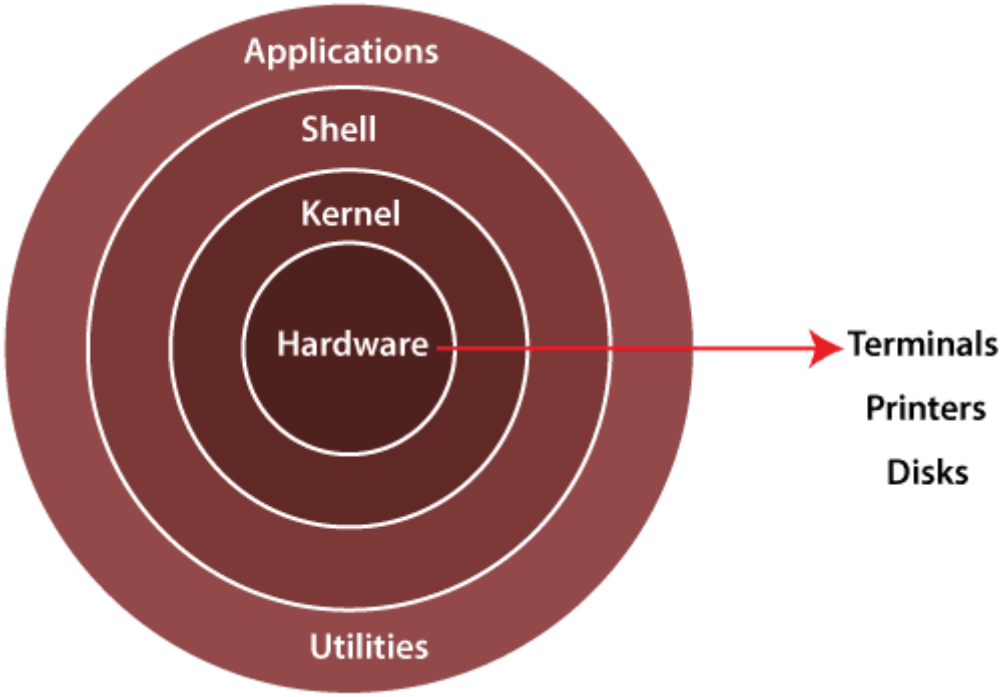
Why is an OS needed?

Abstraction: Applications only need to know how to open and write files via APIs, while the actual interaction with the hardware is handled by the OS.

Resource Management: Fairly shares the CPU and RAM among processes.

Protection and Authorization: Prevents processes from interfering with each other or crashing the system.

Linux is a multitasking, multi-user operating system



Consists of 4 main layers:

Hardware: physical devices

Linux kernel: the core layer that manages resources

Shell: A command interpreter that acts as a bridge, taking commands from the user and passing them to the Kernel (via System Calls).

Applications & Utilities: The user workspace consisting of built-in utilities (ls, cat, grep) and application software installed or written by the user

User space & Kernel space

- *User space*

No direct access to hardware.

Each process has its own isolated virtual memory space; it cannot read the RAM of other processes.

If an application crashes, it does not affect other processes.

- *Kernel space*

Full execution privileges for all instruction sets and chip control.

Full read and write access to all memory locations in RAM.

If an error occurs, the entire system crashes (Kernel Panic/BSOD).

Why must user space and kernel space be separated into two distinct regions?

- System stability: A faulty or crashed application cannot drag down the entire operating system with it.
- Security: Prevents applications from snooping on each other's data or sabotaging the system.
- Hardware protection: Avoids multiple applications from directly interfering with hardware components, which could cause conflicts or damage.

What are the memory differences between User space and Kernel space?

User space

- A separate, completely isolated memory space.
- When an application creates variables and assigns values, everything takes place within its own memory space.
- It cannot access the memory space of other processes.
-

Kernel space

- A shared memory space for the entire system.
- Has read and write access to every memory location, every process, and every device.
- Protected by hardware privilege modes.

The process of the CPU switching between User mode and Kernel mode and vice versa

The mode switch from User to Kernel is entirely managed by hardware through:

- System calls: Functions like open(), read(), write(), fork(), close(), execve() used to request the kernel to perform tasks on behalf of the application.
- Hardware interrupts: External hardware sends electrical signals to the CPU, such as incoming network packets or GPIO button presses.
- Exceptions / Faults

The process of switching from User to Kernel:

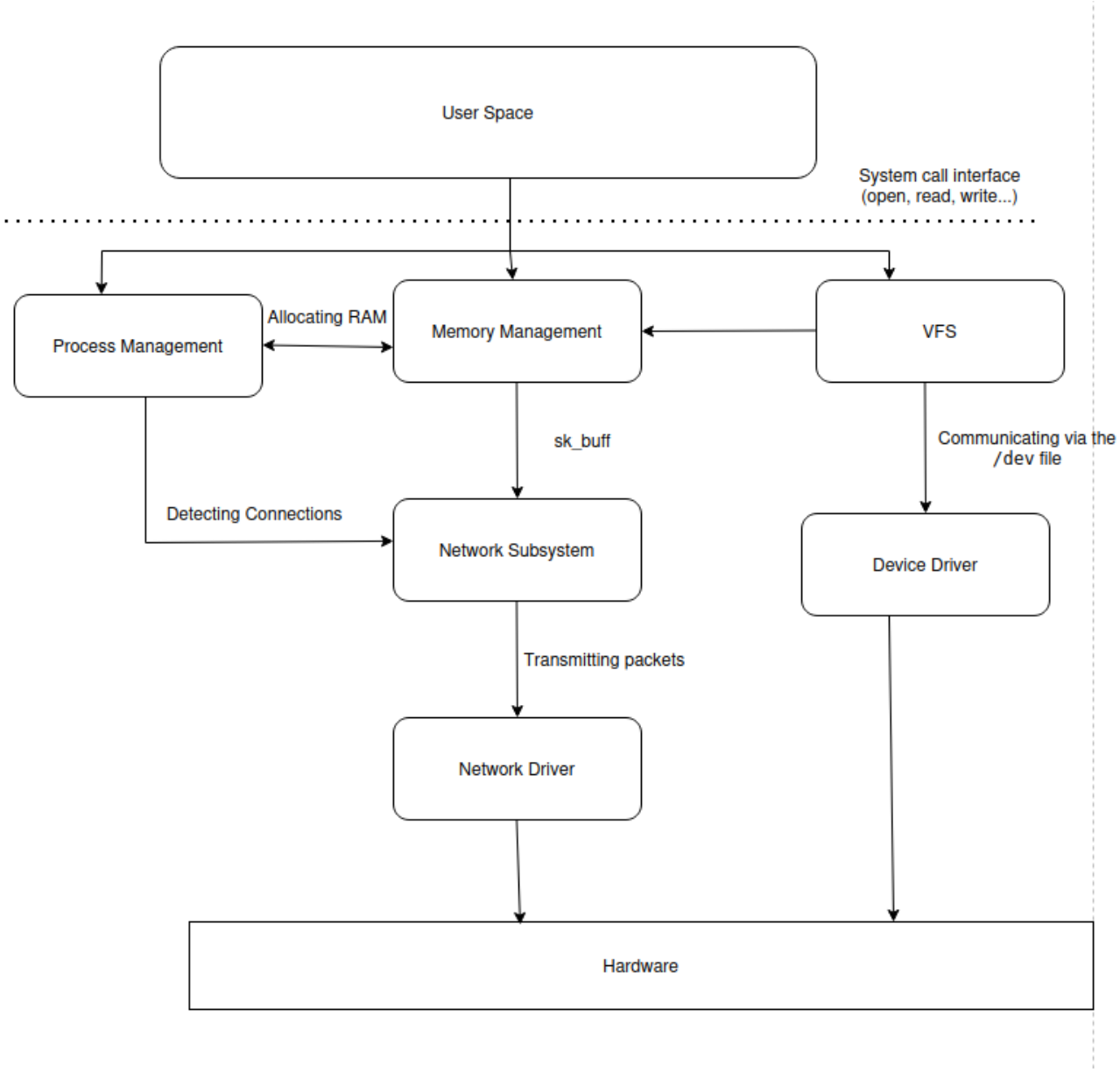
- User mode: The CPU executes instructions within User space.
- Save context: The CPU saves the state, including the Program Counter (instruction pointer) and register variable values, into the Kernel Stack.
- Mode switch: Electronic circuitry in the CPU flips the status bit from User mode to Kernel mode, and the CPU jumps to an address designated by the Kernel.
- Execution: The driver source code runs.
- Return to User mode: After the driver finishes its work, the Kernel executes a specific machine instruction to return; the CPU restor

Linux kernel components

- Process Management: Creating and terminating processes, CPU scheduler allocation, synchronization, and Inter-Process Communication (IPC).
- Memory Management: Managing RAM, creating virtual memory, paging, and dynamic memory allocation (kmalloc/kfree).
- File System (Virtual File System - VFS): Providing a virtual directory tree, managing files and folders -> "Everything is a file"
- Device Drivers: Source code layers that directly interact with peripheral hardware registers.
- Network Subsystem: Collecting, routing, and distributing network packets (TCP/IP).

The kernel components coordinate in a closed loop: Memory Management provides memory space for everything; Process Management coordinates CPU execution rights; the File System acts as a standardized logical interface; Device Drivers and Network subsystems act as the direct hands interacting with external hardware to read and write data.

Lab 1



User space: The restricted sandboxed environment where standard user applications, shell programs, and utilities run without direct hardware access.

System Call Interface (SCI): The secure gateway that allows user-space programs to request services from the kernel (switching from User Mode to Kernel Mode via software interrupts/SVC).

Process Management: Responsible for creating, scheduling, and terminating processes. It uses algorithms (CPU scheduler) to share processor time fairly and efficiently among competing tasks.

Memory Management: Manages physical RAM, implements virtual memory via paging, and handles dynamic memory allocation (kmalloc/kfree) so processes stay isolated and secure.

VFS (Virtual File System): A common software abstraction layer that lets users and programs interact with different storage devices, pipes, sockets, and even hardware devices using standard file operations (open, read, write, close).

Device Driver: Low-level software modules that translate kernel commands into direct hardware operations by writing to device registers, handling interrupts, and managing DMA.

Network Subsystem: Handles protocol stacks (like TCP/IP), packet routing, fragmentation, and queuing to ensure network communication functions smoothly.

Network Driver: The specific driver interfacing directly with the physical network interface card (NIC) hardware chip to send and receive raw electrical or optical signals.

2. Kernel Module/ Driver Basics

Linux Kernel Module (LKM)

- An LKM is a piece of source code (compiled into a .ko file) that can be loaded into or removed from the Kernel at runtime without needing to reboot the system.
- Saves RAM (drivers are only loaded when needed) and accelerates software development (modify driver code -> reload the module without rebooting).

Built-in Module (Compiled directly into the Kernel)	Loadable Kernel Module (LKM)
The module file is compiled and linked directly into the vmlinuz file.	Separate module files in .ko format.
Cannot be unloaded while the system is running.	Can be loaded or unloaded at runtime -> convenient for development and testing.
Typically used for drivers essential for booting up.	Saves RAM (loaded only when needed).
Examples: Network card driver, hard drive driver.	Typically used for: Printer drivers, USB, Wi-Fi, Bluetooth, sound card drivers.

Kernel Driver

- Acts as a bridge between hardware and the user-facing interface:
  - Top layer: Applications, Kernel, VFS (Virtual File System)
  - Bottom layer: Registers, Interrupts, DMA (Direct Memory Access)
- Design Principle - "Mechanism, not Policy": The driver only provides the mechanism (e.g., turning a light on/off), while the policy is determined by the user or application (e.g., how it blinks, how long to keep the light on, etc.).

A simple driver to control basic hardware

```
#include <linux/module.h>
#include <linux/init.h>
#include <linux/fs.h>
#include <linux/cdev.h>
#include <linux/uaccess.h>
```

```
#include <linux/gpio.h>

#define DRIVER_NAME "gpio_lab_driver"
#define BASE_OFFSET 512

static int led_gpios[] =
{
    BASE_OFFSET + 5,   // LED 1: GPIO 5  (517)
    BASE_OFFSET + 6,   // LED 2: GPIO 6  (518)
    BASE_OFFSET + 13,  // LED 3: GPIO 13 (525)
    BASE_OFFSET + 19   // LED 4: GPIO 19 (531)
};
#define NUM_LEDS 4

static dev_t dev_num;
static struct class *lab_class;
static struct cdev led_cdev;

static ssize_t led_write(struct file *filep, const char __user *buf, size_t len, loff_t
*offset)
{
    char kbuf[16] = {0};
    int led_idx, state;

    if (len > sizeof(kbuf) - 1) len = sizeof(kbuf) - 1;
    if (copy_from_user(kbuf, buf, len)) return -EFAULT;

    if (sscanf(kbuf, "%d %d", &led_idx, &state) == 2)
    {
        if (led_idx >= 1 && led_idx <= NUM_LEDS) {
            gpio_set_value(led_gpios[led_idx - 1], state ? 1 : 0);
            pr_info("my_leds: LED %d set thanh %d\n", led_idx, state);
        } else {
            pr_warn("my_leds: So LED khong hop le (chi tu 1 den 4)\n");
        }
    }
    return len;
}

static struct file_operations led_fops =
```

```
{
    .owner = THIS_MODULE,
    .write = led_write,
};

static int __init lab_driver_init(void)
{
    int i, ret;

    ret = alloc_chrdev_region(&dev_num, 0, 1, DRIVER_NAME);
    if (ret < 0) return ret;

    lab_class = class_create(DRIVER_NAME);

    cdev_init(&led_cdev, &led_fops);
    cdev_add(&led_cdev, dev_num, 1);
    device_create(lab_class, NULL, dev_num, NULL, "my_leds");

    for (i = 0; i < NUM_LEDS; i++) {
        gpio_request(led_gpios[i], "GPIO_LED");
        gpio_direction_output(led_gpios[i], 0);
    }

    pr_info("Lab Driver: Da nap driver 4 LED thanh cong!\n");
    return 0;
}

static void __exit lab_driver_exit(void)
{
    int i;

    for (i = 0; i < NUM_LEDS; i++)
    {
        gpio_set_value(led_gpios[i], 0);
        gpio_free(led_gpios[i]);
    }

    device_destroy(lab_class, dev_num);
    cdev_del(&led_cdev);
    class_destroy(lab_class);
```

```
    unregister_chrdev_region(dev_num, 1);

    pr_info("Lab Driver: Da go bo driver 4 LED!\n");
}

module_init(lab_driver_init);
module_exit(lab_driver_exit);

MODULE_LICENSE("GPL");
MODULE_AUTHOR("Huyen");
MODULE_DESCRIPTION("Driver 4 LEDs (GPIO 5, 6, 13, 19) for Raspberry Pi 4");
```

Data flow

User space

- Open the shell -> The shell is a user program -> The shell has a UID.
- Type a command -> The 'tee' process (running with root privileges) executes:

```
fd = open("/dev/my_leds", O_WRONLY);
write(fd, "1 1", 3);
```

- The system call triggers the SVC (Supervisor call) instruction -> Switches from user mode to kernel mode.

Virtual File System (VFS)

- The kernel receives the request to write to the file /dev/my_leds.
- The kernel identifies the driver and calls the function led_write(filep, buf, len, offset).
- The pointer buf points to the user space RAM containing "1 1\n".

Device driver (gpio_lab_driver.c)

- copy_from_user(); -> Copies the string "1 1\n" from user space to kernel space (kbuf).
- Uses sscanf(kbuf, "%d %d", &led_idx, &state) -> Parses out: led_idx = 1 (LED 1), state = 1 (ON).
- Retrieves the corresponding pin: led_gpios[0] = 517 (GPIO 5 on Pi 4).
- Calls the kernel function: gpio_set_value(517, 1);
- Writes the value 1 to the register.

MMU & Register

- The function gpio_set_value operates on the kernel virtual address of the GPIO registers.
- The MMU chip translates the virtual address into a physical address.
- Bit number 5 of the physical control register is set to 1.

Hardware

- GPIO pin number 5 is supplied with a 3.3V voltage -> The LED turns on.

What is the difference between an executable file and a .ko file?

C Standard Library

- User space: Supported by the standard C library (glibc).
- Kernel space (.ko): No standard C library support is available.

Lifecycle

- User App: Executes instructions inside main() $\rightarrow$ calls return 0 to terminate and release the process.
- Kernel Module (.ko): The module resides permanently in RAM and only disappears when the rmmod command is invoked.

Consequences of Errors (Crashes)

- User space: Protected by the MMU (Memory Management Unit); only the crashing process is terminated.
- .ko file: Runs with full root/kernel privileges $\rightarrow$ encountering an error causes a system crash (Kernel Panic).

Revision #3
Created 2026-10-13 10:32:01 UTC by Huyen Nguyen
Updated 2026-10-14 10:34:51 UTC by Huyen Nguyen